

Test eksploracyjny nr 1- Wyszukiwarka biletów Intercity

Cel: Odnalezienie potencjalnych problemów w module wyszukiwania i rezerwacji biletów

Czas trwania: 10 minut

Zakres: pola „stacja początkowa” , „stacja końcowa” , data , przyciski , liczba pasażerów dorosłych i dzieci

- Maksymalna liczba pasażerów w pierwszej oraz drugiej klasie wynosi 6.
- Przy 6 dorosłych nie można dodać biletu dziecięcego.
- Brak informacji o tym ograniczeniu może wprowadzać użytkownika w błąd.

Aktualny rezultat: System blokuje dodanie dziecka i nie wyświetla żadnego komunikatu informującego o przyczynie.

Oczekiwany rezultat: Po dodaniu 6 dorosłych i próbie dodania dziecka system powinien wyświetlić komunikat: „Maksymalna liczba pasażerów wynosi 6”.

Wnioski: Użytkownik nie rozumie,dlaczego nie może dodać dziecka do rezerwacji, co może prowadzić do porzucenia biletu.

Severity: Medium

Priority: Medium

Projekt: Testy strony logowania – aplikacja bankowa X

Opis projektu:

Celem testów było sprawdzenie poprawności działania modułu logowania w aplikacji bankowej. Skupiono się na poprawności działania pól „Login” i „Hasło”, przycisku „Zaloguj”, komunikatach błędów oraz funkcjonowaniu sesji użytkownika.

Typ testów: Checklista (testy manualne)

Tabela testów:

Checklista testów – Moduł logowania (Aplikacja bankowa X)

ID	Element	Sprawdzenie	Status
CH-01	Pole „Login”	Czy przyjmuje poprawny login	PASS
CH-02	Pole „Login”	Czy odrzuca pusty login	PASS
CH-03	Pole „Login”	Czy odrzuca login z niedozwolonymi znakami	PASS
CH-04	Pole „Login”	Czy przyjmuje maksymalną długość loginu	PASS
CH-05	Pole „Hasło”	Czy hasło jest maskowane	PASS
CH-06	Pole „Hasło”	Czy odrzuca puste hasło	PASS

ID	Element	Sprawdzenie	Status
CH-07	Pole „Hasło”	Czy przyjmuje maksymalną długość hasła	PASS
CH-08	Pole „Hasło”	Czy przyjmuje znaki specjalne	PASS
CH-09	Przycisk „Zaloguj”	Czy działa po wpisaniu poprawnych danych	PASS
CH-10	Przycisk „Zaloguj”	Czy nie działa przy pustym loginie	PASS
CH-11	Przycisk „Zaloguj”	Czy nie działa przy pustym hasle	PASS
CH-12	Przycisk „Zaloguj”	Czy reaguje na enter w polu hasła	PASS
CH-13	Komunikat błędu	Czy wyświetla się przy złym hasle	FAIL
CH-14	Komunikat błędu	Czy wyświetla się przy nieistniejącym loginie	FAIL
CH-15	Komunikat błędu	Czy jest zrozumiały dla użytkownika	PASS
CH-16	Link „Zapomniałem hasła”	Czy przekierowuje do resetu hasła	PASS
CH-17	Link „Rejestracja”	Czy przekierowuje do formularza rejestracji	PASS
CH-18	Pole „Hasło”	Czy można włączyć widoczność hasła	PASS
CH-19	Pole „Login”	Czy autofocus działa po wejściu na stronę	PASS
CH-20	Sesja	Czy użytkownik zostaje zalogowany po odświeżeniu strony	PASS
CH-21	Sesja	Czy po wylogowaniu sesja jest zakończona	PASS
CH-22	Pola formularza	Czy widać etykiety pól	PASS
CH-23	Responsywność	Czy strona działa na telefonie	PASS
CH-24	Responsywność	Czy strona działa na tablecie	PASS
CH-25	Przycisk „Zaloguj”	Czy zmienia stan po kliknięciu	PASS

Wnioski:

1. Strona logowania działa poprawnie w większości scenariuszy.
2. Zauważono nieprawidłowe komunikaty przy błędnym hasle i nieistniejącym loginie (FAIL).
3. Testy pozwoliły zidentyfikować obszary wymagające poprawy oraz sprawdzić podstawową funkcjonalność i użyteczność modułu logowania.

Zakres testów:

- pola: login, hasło
- walidacje
- responsywność
- sesja użytkownika

Środowisko testowe:

- Przeglądarka: Chrome
- System: Windows

- Rozdzielczości testowe: 1920x1080

Łącznie wykonano 25 testów, z czego 23 zakończyły się PASS, a 2 FAIL (komunikaty błędów wymagają poprawy).

Projekt: Testowanie API – FakeStoreAPI (Postman)

Opis projektu:

W ramach testów manualnych wykorzystałem publiczne API FakeStoreAPI. Celem było przetestowanie podstawowych endpointów sklepu testowego: pobieranie listy produktów (GET) oraz dodawanie nowego produktu (POST). Sprawdzano poprawność odpowiedzi, strukturę danych oraz obsługę błędów.

Narzędzie: Postman

Tabela testów API

ID	Endpoint	Zapytanie	Sprawdzenie	Status
API-01	/products	GET	Status 200 OK, zwraca listę produktów	PASS
API-02	/products	GET	Każdy produkt zawiera pola: id, title, price, category, description, image	PASS
API-03	/products	POST	Dodanie nowego produktu z poprawnymi danymi	PASS
API-04	/products	POST	Status odpowiedzi 201, zwraca nowy produkt z unikalnym ID	PASS
API-05	/products	POST	Wysłanie zapytania bez wymaganych pól (np. brak title)	FAIL
API-06	/products	GET	Sprawdzenie, czy nowy produkt pojawia się na liście produktów	PASS

Opis wykonanych testów w Postmanie

1. **GET /products** – pobranie listy wszystkich produktów. Sprawdzono:
 - status odpowiedzi (200 OK)
 - poprawność struktury danych JSON
 - zawartość wymaganych pól (id, title, price, category, description, image)
2. **POST /products** – dodanie nowego produktu. Sprawdzono:
 - wysyłanie poprawnych danych
 - status odpowiedzi (201 Created)
 - czy nowy produkt zwracany w odpowiedzi ma unikalne id
 - czy nowy produkt pojawia się w GET /products

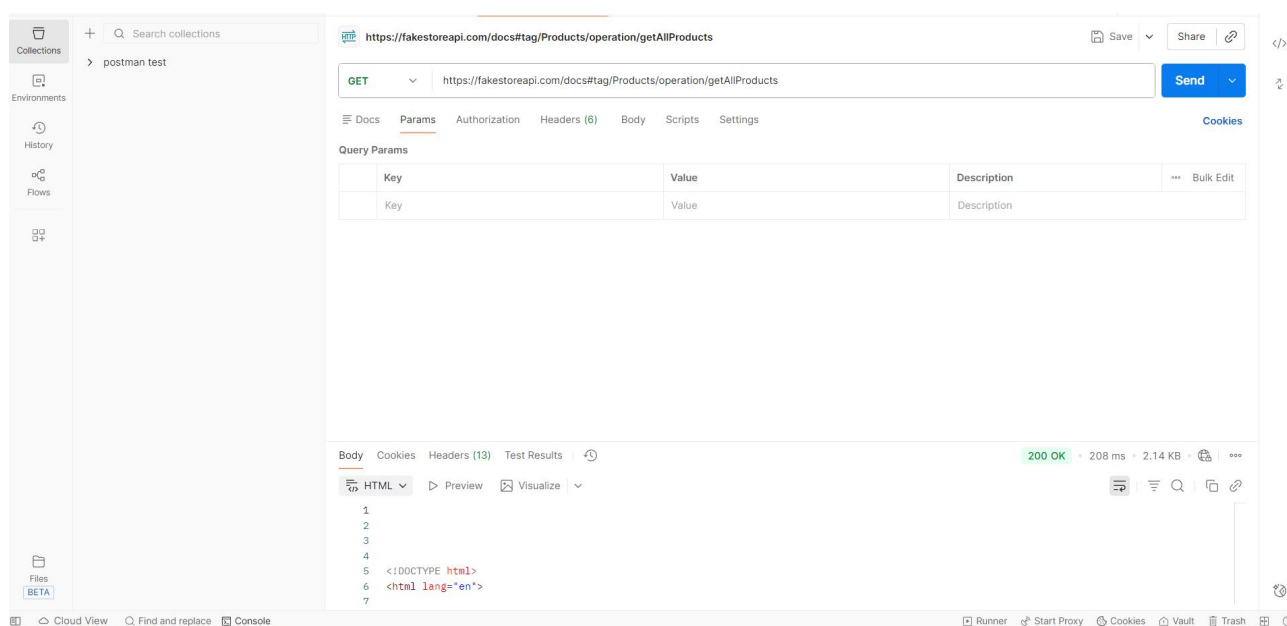
3. Test negatywny (POST /products) – wysłanie zapytania bez wymaganych pól, aby sprawdzić reakcję API.

Wnioski

- API poprawnie obsługuje standardowe zapytania GET i POST.
- Test negatywny wykazał brak obsługi niepełnych danych (FAIL) – istotna informacja dla deweloperów.
- Portfolio pokazuje praktyczne umiejętności: tworzenie i wykonywanie zapytań w Postmanie, weryfikację odpowiedzi API oraz dokumentowanie wyników testów.
- Testy GET i POST pozwalają zweryfikować zarówno pozytywne, jak i negatywne scenariusze działania API.

API nie zwraca czytelnego błędu przy niepełnych danych (FAIL). W realnym systemie może prowadzić do niekontrolowanych wyjątków lub błędów backendu. 1 – Poprawny status odpowiedzi pobranej listy produktów

Screenshot 1 - GET /products (lista produktów)



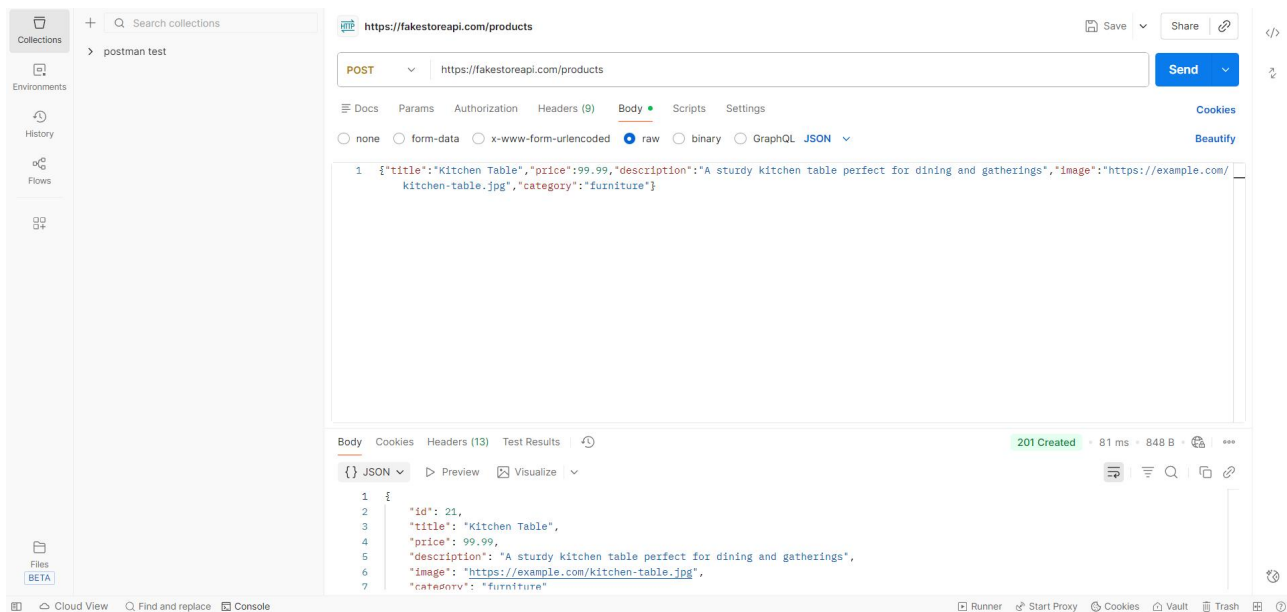
Na screenie widoczna jest odpowiedź serwera na zapytanie GET wysłane do endpointu /products.

Status odpowiedzi: **200 OK**

Otrzymane dane mają poprawną strukturę JSON i zawierają wymagane pola (id, title, price, description, category, image).

Test potwierdza, że API poprawnie zwraca listę dostępnych produktów.

Screenshot 2 – POST /products (dodanie produktu)



Widoczny jest wynik wysłania zapytania POST z poprawnym body JSON.

Status odpowiedzi: **201 Created**

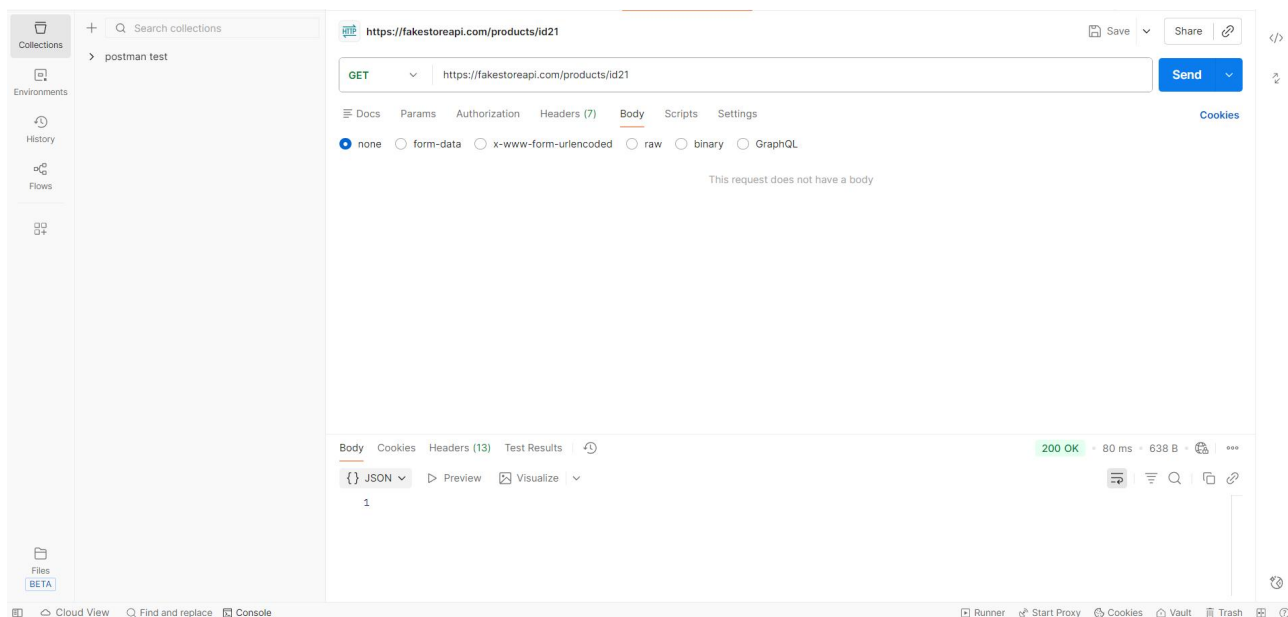
API zwróciło nowo utworzony produkt wraz z automatycznie wygenerowanym polem **id**.

Test potwierdza, że endpoint poprawnie obsługuje dodawanie nowych produktów.

Body request (POST /products)

```
{
  "title": "Stół kuchenny",
  "price": 99.99,
  "description": "Duży stół drewniany",
  "category": "kitchen",
  "image": "https://example.com/stol.jpg"
}
```

Screenshot 3 – Pobranie dodanego produktu po ID (GET /products/{id})



Screen przedstawia zapytanie GET o produkt o numerze ID wygenerowanym podczas testu POST.

Status odpowiedzi: **200 OK**

Zwrotka serwera zawiera te same dane, które wcześniej wysłałem w zapytaniu POST.

Test potwierdza, że produkt został poprawnie zapisany i jest dostępny pod swoim ID.

Przykład JSON losowo pobranego produktu:

```
{  
  "id": 1,  
  "title": "Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops",  
  "price": 109.95,  
  "description": "Your perfect pack for everyday use and walks in the forest. Stash your laptop  
(up to 15 inches) in the padded sleeve, your everyday",  
  "category": "men's clothing",  
  "image": "https://fakestoreapi.com/img/81fPKd-2AYL. AC SL1500 t.png"  
}
```

Zgłoszenie błędu – JIRA (przykład)

ID: BUG-001

Tytuł: Brak informacji o limicie pasażerów w pierwszej oraz drugiej klasie

Środowisko:

- Strona: intercity.pl
- Chrome , Windows 11

Kroki do odtworzenia:

1. Otwórz wyszukiwarkę biletów
2. Ustaw „2 klasa” oraz „1 klasa”
3. Dodaj 6 dorosłych
4. Spróbuj dodać dziecko

Rzeczywisty rezultat:

System blokuje dodanie dziecka, ale nie informuje o przyczynie.

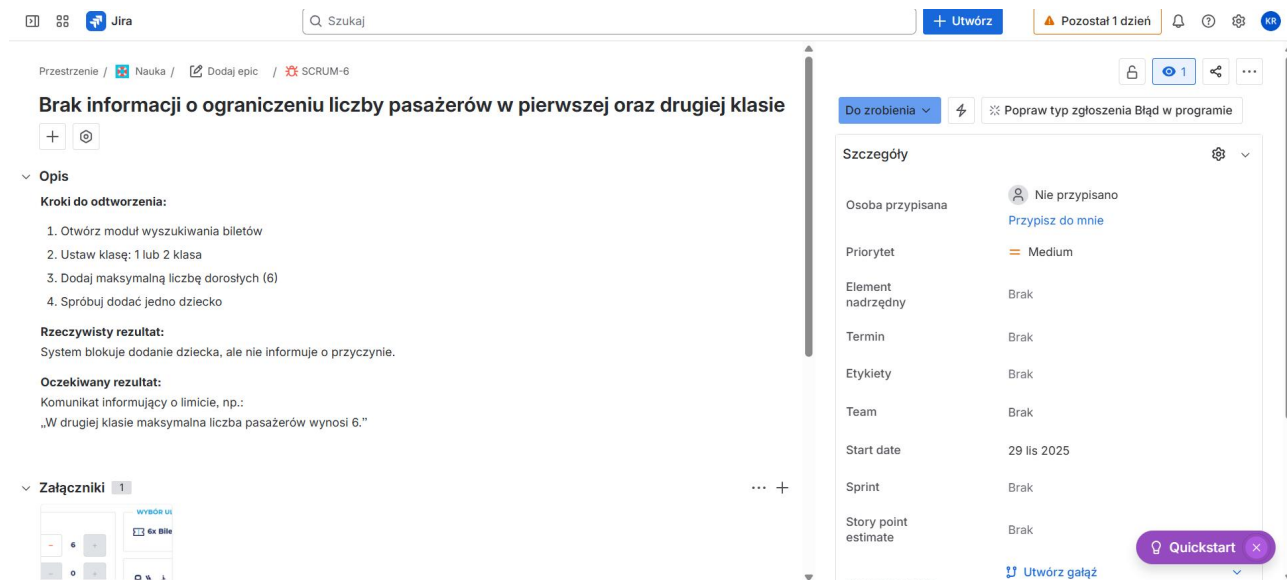
Oczekiwany rezultat:

Powinien pojawić się komunikat o limicie pasażerów.

Priorytet: Medium

Severity: Medium

Załącznik: screenshot.jpg



LICZBA PODRÓŻUJĄCYCH

Dorośli

6

Dzieci

0

WYBÓR ULG

6x Bilet Normalny

[ZMIEN](#)

[Filtry](#)

[ZMIEN](#)

